

MySQL Practice Assignment

Joins, Subqueries, Constraints & Normalization

Name:	Siddhesh Thakare	Platform:	Programiz Online SQL Compiler
Topic:	Joins, Subqueries, Constraints, Normalization, DDL, DML, DQL	Status:	Completed

Reference Database Schema

Table	Column	Data Type	Key / Constraint
Customers	customer_id	INT	PRIMARY KEY
Customers	first_name	VARCHAR(100)	-
Customers	last_name	VARCHAR(100)	-
Customers	age	INT	-
Customers	country	VARCHAR(100)	-
Orders	order_id	INT	PRIMARY KEY
Orders	item	VARCHAR(100)	-
Orders	amount	INT	-
Orders	customer_id	INT	FOREIGN KEY
Shippings	shipping_id	INT	PRIMARY KEY
Shippings	status	VARCHAR(100)	-
Shippings	customer	INT	FOREIGN KEY

Part 1: Multiple Choice Questions

Question 1: DDL vs. DML — TRUNCATE vs. DELETE

Which statement correctly differentiates TRUNCATE TABLE from DELETE FROM table_name in MySQL?

Answer: B) TRUNCATE is a DDL command that removes data by deallocating data pages without firing individual row-level triggers, whereas DELETE is a DML command that deletes records row-by-row and records each deletion in the transaction log.

Question 2: Foreign Key Constraints & Cascading Actions

Given Orders with a Foreign Key referencing Customers(customer_id) ON DELETE SET NULL, what happens when customer_id = 5 is deleted?

Answer: C) Customer 5 is deleted, and the customer_id column in Orders for matching rows is set to NULL.

Question 3: Emulating FULL OUTER JOIN in MySQL

Which combination of SQL constructs produces the exact functional equivalent of a FULL OUTER JOIN?

Answer: B) `SELECT * FROM TableA LEFT JOIN TableB ON TableA.id = TableB.id UNION SELECT * FROM TableA RIGHT JOIN TableB ON TableA.id = TableB.id;`

Question 4: Subqueries — EXISTS vs. IN Operator

How does MySQL process a correlated subquery using EXISTS?

Answer: B) It evaluates the subquery for each row of the outer table and terminates the inner scan as soon as the first matching order record is found.

Question 5: Database Normalization — 3NF Definition

A database table is in 3NF if and only if it satisfies which set of conditions?

Answer: B) It is in 2NF and every non-key column is directly dependent on the primary key (no transitive dependencies).

Question 6: DQL Execution Order — WHERE vs. HAVING

In a DQL statement containing WHERE, GROUP BY, and HAVING, in what order are these evaluated?

Answer: C) `WHERE → GROUP BY → HAVING`

Question 7: MySQL Constraints — UNIQUE Key with NULL Values

How does a column defined with a UNIQUE constraint handle NULL values in InnoDB?

Answer: B) It allows multiple NULL entries because NULL represents an unknown value and is not considered equal to another NULL.

Question 8: DDL ALTER TABLE Operations

Which command adds a new column birth_date of type DATE with NOT NULL to Customers?

Answer: B) `ALTER TABLE Customers ADD birth_date DATE NOT NULL;`

Question 9: Self Join Concepts

Which business scenario requires a Self Join?

Answer: B) Finding employees and the names of their direct managers when both employees and managers are stored in the same Employees table.

Question 10: DML UPDATE with Subqueries

What happens when UPDATE ... WHERE customer_id = (subquery) is run and the subquery returns 3 rows?

Answer: B) MySQL will throw an error because the single-row comparison operator = is used with a subquery that returns multiple rows.

Part 2: Hands-On Practical SQL Tasks

Task 1: Multi-Table DQL Aggregation with INNER JOIN

Objective: Calculate the total amount spent by each customer across all their orders. Display first_name, last_name, and total_spent, sorted by total_spent in descending order.

```
SELECT
  c.first_name,
  c.last_name,
  SUM(o.amount) AS total_spent
FROM Customers c
INNER JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_spent DESC;
```

Result: Query executed successfully on Programiz — returns each customer's aggregated spend, ordered from highest to lowest.

Task 2: Unmatched Records Identification using LEFT JOIN

Objective: Identify all customers who have never placed an order. Display their first_name, last_name, and country.

```
SELECT
  c.first_name,
  c.last_name,
  c.country
FROM Customers c LEFT JOIN Orders o ON
  c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Result: The LEFT JOIN keeps all customers, and filtering on o.order_id IS NULL isolates the ones with no matching order row.

Task 3: Filtering via Scalar Subquery

Objective: Retrieve all orders where the order amount is strictly greater than the overall average order amount. Display order_id, item, amount, and customer_id.

```
SELECT
order_id,
item,
amount,
customer_id
FROM Orders
WHERE amount > (
SELECT AVG(amount)
FROM Orders
);
```

Result: The inner query computes a single scalar value (the average), which the outer query then uses as a comparison threshold.

Task 4: Correlated Subquery using EXISTS

Objective: Retrieve first_name and last_name of customers who have at least one shipping record with status 'Pending'.

```
SELECT
c.first_name,
c.last_name
FROM Customers c
WHERE EXISTS (
SELECT 1
FROM Shippings s
WHERE s.customer = c.customer_id
AND s.status = 'Pending'
);
```

Result: EXISTS confirms membership without needing to return actual data, making the correlated check efficient.

Task 5: Grouped Filtering using GROUP BY & HAVING

Objective: Display each country with the average customer age, filtered to countries with more than 1 customer and average age greater than 20.

```
SELECT country,  
AVG(age) AS avg_age  
FROM Customers  
GROUP BY country HAVING  
COUNT(customer_id) > 1  
AND AVG(age) > 20;
```

Result: HAVING is used (rather than WHERE) because the filter conditions apply to aggregated group-level results.

Task 6: DDL Table Creation with Constraints (Manual Primary Key)

Objective: Create a VIP_Memberships table without AUTO_INCREMENT, including a foreign key with cascading deletes, a default value, and a CHECK constraint.

```
SELECT country,  
AVG(age) AS avg_age  
FROM Customers  
GROUP BY country HAVING  
COUNT(customer_id) > 1  
AND AVG(age) > 20;
```

Result: Table created successfully — vip_id is manually assigned as the primary key, and ON DELETE CASCADE ensures membership rows are removed automatically if the related customer is deleted.

Task 7: Dynamic DML Insertion without AUTO_INCREMENT

Objective: Insert a new order for customer 'John Doe' (item = 'Webcam', amount = 150), generating the next order_id dynamically.

```
INSERT INTO Orders (order_id, item, amount, customer_id)  
SELECT COALESCE(MAX(order_id),  
0) + 1,  
'Webcam',  
150, (SELECT customer_id FROM Customers WHERE first_name = 'John' AND last_name =  
'Doe' LIMIT 1)  
FROM Orders;
```

Result: COALESCE(MAX(order_id), 0) + 1 safely generates the next primary key value even on an empty table, and the nested subquery resolves the customer_id for 'John Doe'.

Task 8: Conditional DML Update with Subquery

Objective: Increase the amount by 10% for all orders placed by customers residing in the USA.

```
UPDATE Orders
SET amount = amount * 1.10
WHERE customer_id IN (
SELECT customer_id
FROM Customers WHERE country = 'USA'
);|
```

Result: IN is used instead of '=' because the subquery can return multiple customer_id values for the USA.

Task 9: Multi-Table Joins (3 Tables)

Objective: Join Customers, Orders, and Shippings to display first_name, last_name, item, amount, and shipping status, keeping orders even if shipping is unassigned.

```
SELECT
c.first_name,
c.last_name,
o.item,
o.amount, COALESCE(s.status, 'Not Shipped') AS
shipping_status
FROM Customers c
INNER JOIN Orders o ON c.customer_id = o.customer_id
LEFT JOIN Shippings s ON c.customer_id = s.customer;|
```

Result: INNER JOIN links customers to their orders, while LEFT JOIN preserves orders without a shipping record; COALESCE substitutes a readable label for missing shipping status.

Task 10: Database Normalization (Refactoring to 3NF without AUTO_INCREMENT)

Objective: Convert the unnormalized Unnormalized_Sales table (customer_id, customer_name, customer_country, order_id, order_item, order_amount) into 3NF.

```
-- Step 1: Normalized Customers table
(3NF) CREATE TABLE Customers_Norm (
customer_id INT PRIMARY KEY,
customer_name VARCHAR(100) NOT NULL,
customer_country VARCHAR(100) );
```

```
-- Step 2: Normalized Orders table referencing Customers_Norm
(3NF) CREATE TABLE Orders_Norm ( order_id INT PRIMARY KEY,
order_item VARCHAR(100) NOT NULL,
order_amount DECIMAL(10,2) CHECK (order_amount >= 0), customer_id INT NOT
NULL, FOREIGN KEY (customer_id) REFERENCES Customers_Norm(customer_id) ON DELETE
CASCADE
);|
```

Result: Splitting the table removes the transitive dependency of customer_name/customer_country on order_id, leaving each non-key attribute dependent only on its own table's primary key — satisfying 3NF